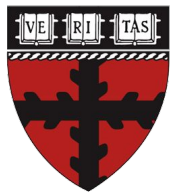
Lecture 15: ML Workflows with Vertex AI

AC215/E115

Pavlos Protopapas
SEAS/Harvard



Announcements

MS3 updates

We have set a peer and self evaluation on canvas. This is a way to assess your contribution for fair work.

1 point - Peer and Self Evaluation

4 points - Adjustment based on Peer, Self and TF Evaluation (contribution, meeting attendance)

20 points - Final presentation, evaluated on:

- Technical Depth
- Business Acumen
- Clarity and Structure
- Visual Communication
- Engagement and Delivery

Submission

Upload your **slides on Canvas by noon on Tuesday**.

Announcements

⚙️ Resources

- Free GPUs, models, and LLM credits available for AIxBio projects.



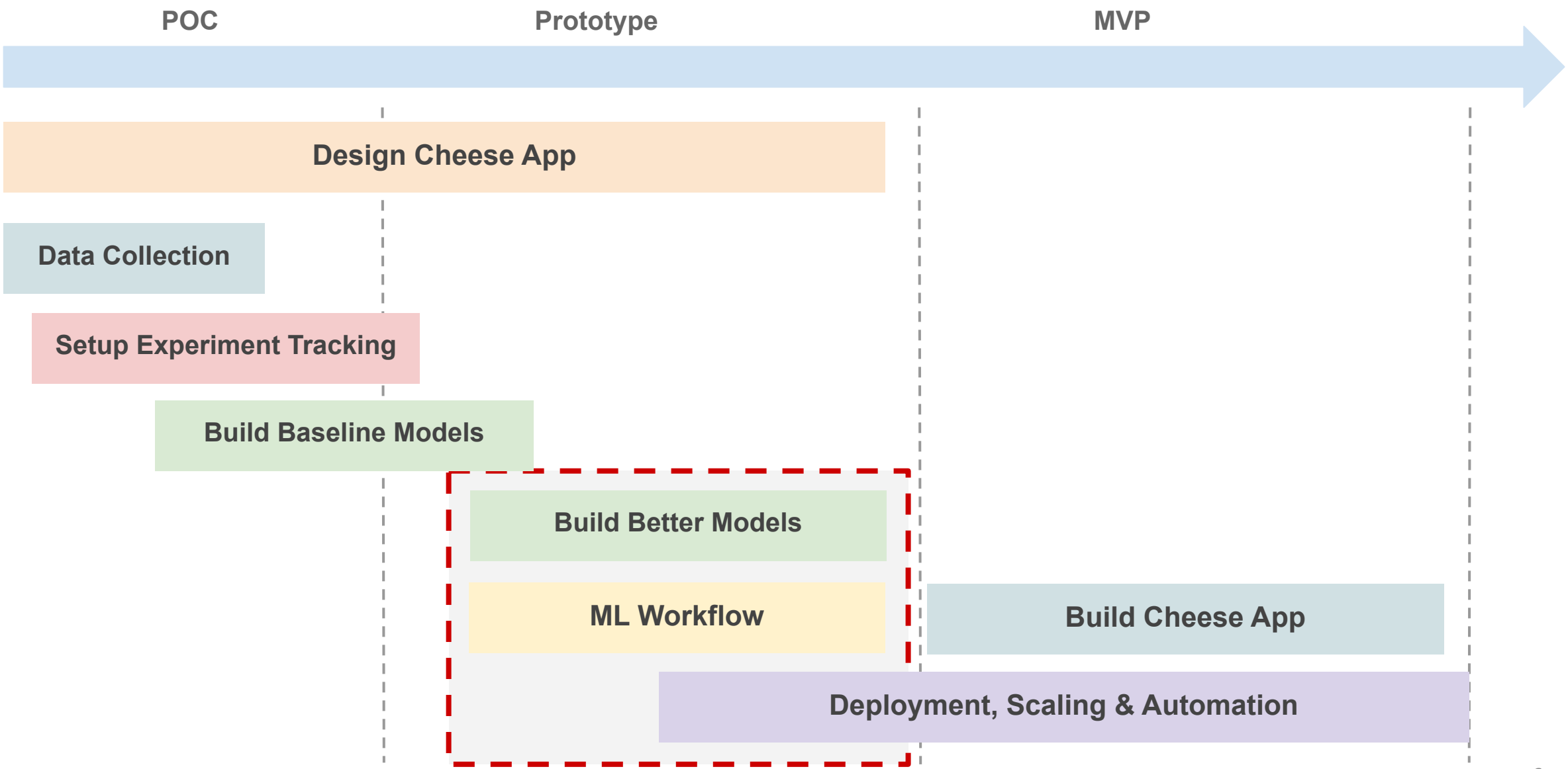
Outline

1. Recap
2. Serverless: Model Deployment with Vertex AI Online Prediction
3. ML Workflow Management
   - Vertex AI Pipelines

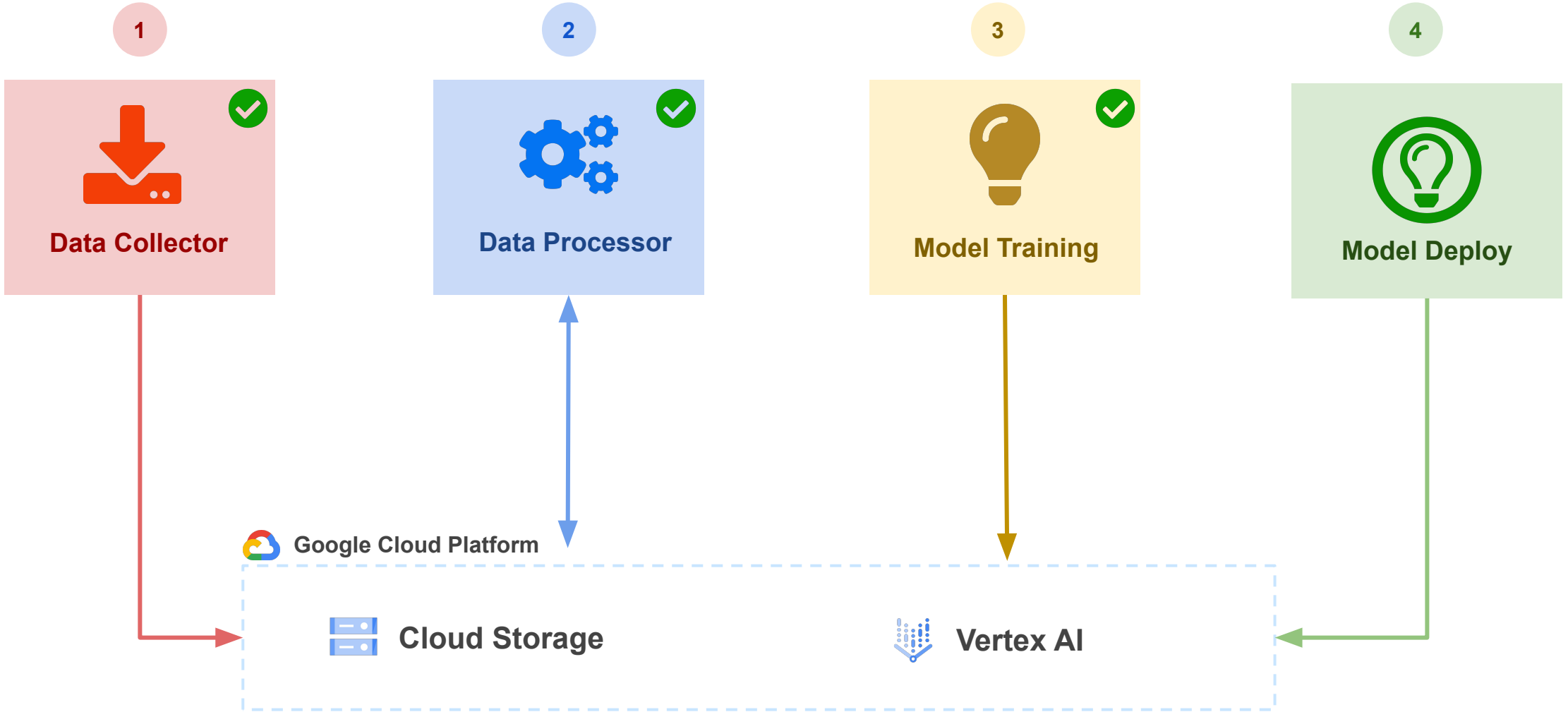
Outline

1. **Recap**
2. Serverless: Model Deployment with Vertex AI Online Prediction
3. ML Workflow Management
   - Vertex AI Pipelines

Review: Cheese App Status



Cheese App Development



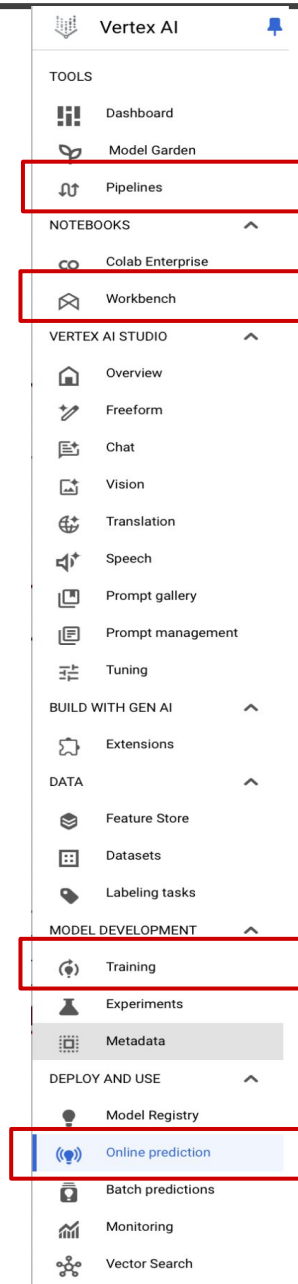
Recap

1. Serverless: Cloud Functions
   - Small, event-driven tasks
   - Great for automating steps like data prep
2. Serverless: Cloud Runs
   - Runs containerized apps
   - Used to serve models or APIs

Outline

1. Recap
2. **Serverless: Model Deployment with Vertex AI Online Prediction**
3. ML Workflow Management
   – Vertex AI Pipelines

Vertex AI



Build ML Workflow

JupyterLab notebooks

Training pipelines and jobs

Serveless deployment

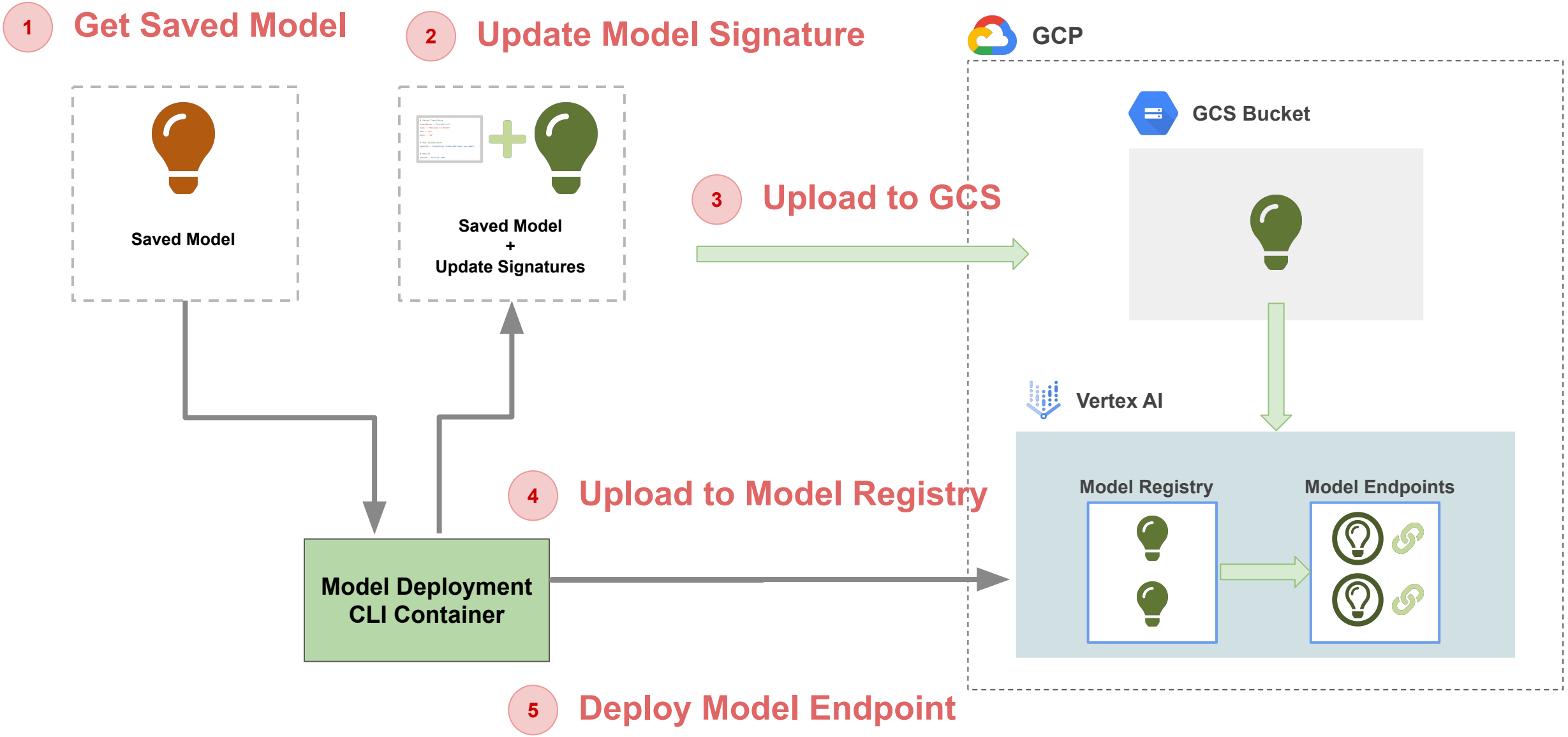
Serverless Model Deployment with **Vertex AI Online Prediction**

What is serverless model deployment?

- Deploy our trained model for predictions with no servers.
- Setup **online** or **batch** prediction modes
- For **online** predictions there is an ongoing cost
- Access GPU or CPU hardware for inference
- Scale out easily
- Alert: Continuous cost to keep endpoint up



Serverless Model Deployment



Serverless Model Deployment: Model Signature

Why do we need to update the model signature?

In development (notebooks):

- You chain steps manually (resize, normalize, etc.).
- You do everything step by step — one cell for resizing, another for normalization.

In production (Vertex AI endpoint):

- Accept **raw image bytes** at the endpoint (no client-side prep).
  Move preprocessing into the serving function so it runs on the server for every request.
- You send one request to the endpoint, and it handles everything.
- Centralized changes (resize/normalize logic) = easier rollback/versioning.

💡 *Client-side brittleness = every client preprocesses data slightly differently.*

Bottom line: Update the signature so the endpoint does **preprocessing + inference** in one place — reliably and at scale.

Serverless Model Deployment: Update Model Signature

```python
# Preprocess Image
def preprocess_image(bytes_input):
    resized = ...
    return resized

# Define tf functions
@tf.function(input_signature=[tf.TensorSpec([None], tf.string)])
def preprocess_function(bytes_inputs):
    decoded_images = tf.map_fn(
        preprocess_image, bytes_inputs, dtype=tf.float32, back_prop=False
    )
    return {"model_input": decoded_images}

@tf.function(input_signature=[tf.TensorSpec([None], tf.string)])
def serving_function(bytes_inputs):
    images = preprocess_function(bytes_inputs)
    results = model_call(**images)
    return results

# Update model signature and save
tf.saved_model.save(
    prediction_model,...,
    signatures={"serving_default": serving_function},
)
```

Define preprocessing function

Define @tf.function for new model signature

Save Model with the new model signature

Tutorial: Serverless Model Deployment

Steps to perform **Serverless Model Deployment** on cheese classification model:

- Create a GCS bucket to store saved model.
- Update Model Serving Signature
- Upload Model to Vertex AI Model Registry.
- Deploy Model as an Endpoint.
- For detailed instructions, please refer to the following link
  - Serverless Model Deployment. (https://github.com/dlops-io/model-deployment)
  - View Model Endpoints. (https://console.cloud.google.com/vertex-ai/online-prediction/endpoints)
  - View Model Registry. (https://console.cloud.google.com/vertex-ai/models)



Vertex AI Online Prediction vs. Cloud Run for Model Deployment

- **Purpose**
  Vertex AI: Designed for ML models — managed, optimized, and monitored.
  Cloud Run: General-purpose — runs any container, not ML-specific.
- **Ease of Use**
  Vertex AI: Direct model deployment with minimal setup.
  Cloud Run: Requires Docker image and manual configuration.
- **Scaling & Monitoring**
  Vertex AI: Built-in autoscaling, model versioning, and drift detection.
  Cloud Run: Scales on HTTP traffic, needs custom ML monitoring.
- **Flexibility**
  Vertex AI: Works best when your goal is **serving models**.
  Cloud Run: Best for **custom apps** that may include ML models.
- **Best Practice**
  Use *Vertex AI* for serving trained models in production.
  Use *Cloud Run* for integrating ML into broader applications or pipelines.

Vertex AI Online Prediction vs. Cloud Run for Model Deployment

Feature	Vertex AI Online Prediction	Cloud Run
Purpose	Managed ML model serving for real-time predictions	General-purpose serverless platform for containerized apps
Ease of Deployment	Deploys models directly, minimal setup	Requires containerization, more configuration
Infrastructure	Fully managed, ML-specific infrastructure with autoscaling and load balancing	Autoscaling serverless platform, not specific to ML
Model Monitoring	Built-in model performance monitoring and data drift detection	No built-in ML monitoring; requires custom implementation
Scaling	Optimized for real-time ML workloads, automatic scaling for high availability	Serverless scaling based on HTTP traffic
Model Versioning	Supports versioning, A/B testing for models	No native support for model versioning
Batch Prediction Support	Yes, with both batch and online prediction capabilities	No specific batch prediction feature; needs custom setup
Framework Support	TensorFlow, PyTorch, XGBoost, and other ML frameworks	Framework-agnostic, supports any containerized code
Security and Permissions	Integrated with IAM for granular access control	Integrated with IAM but without ML-specific controls
Best For	Real-time ML model serving with minimal infrastructure management	Custom APIs and services that may include ML models

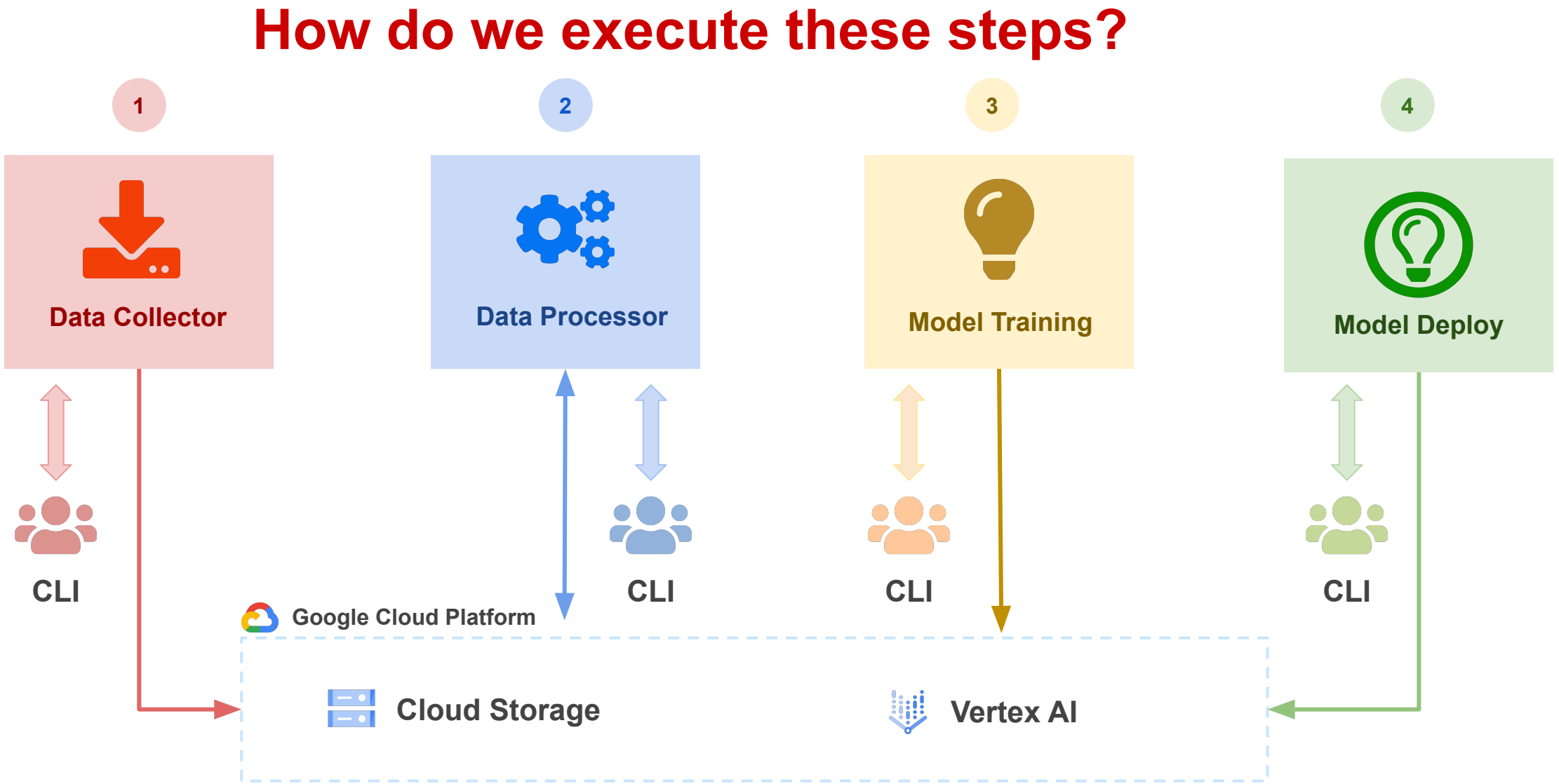
Outline

1. Recap
2. Serverless: Model Deployment
3. **ML Workflow Management**
   - Vertex AI Pipelines

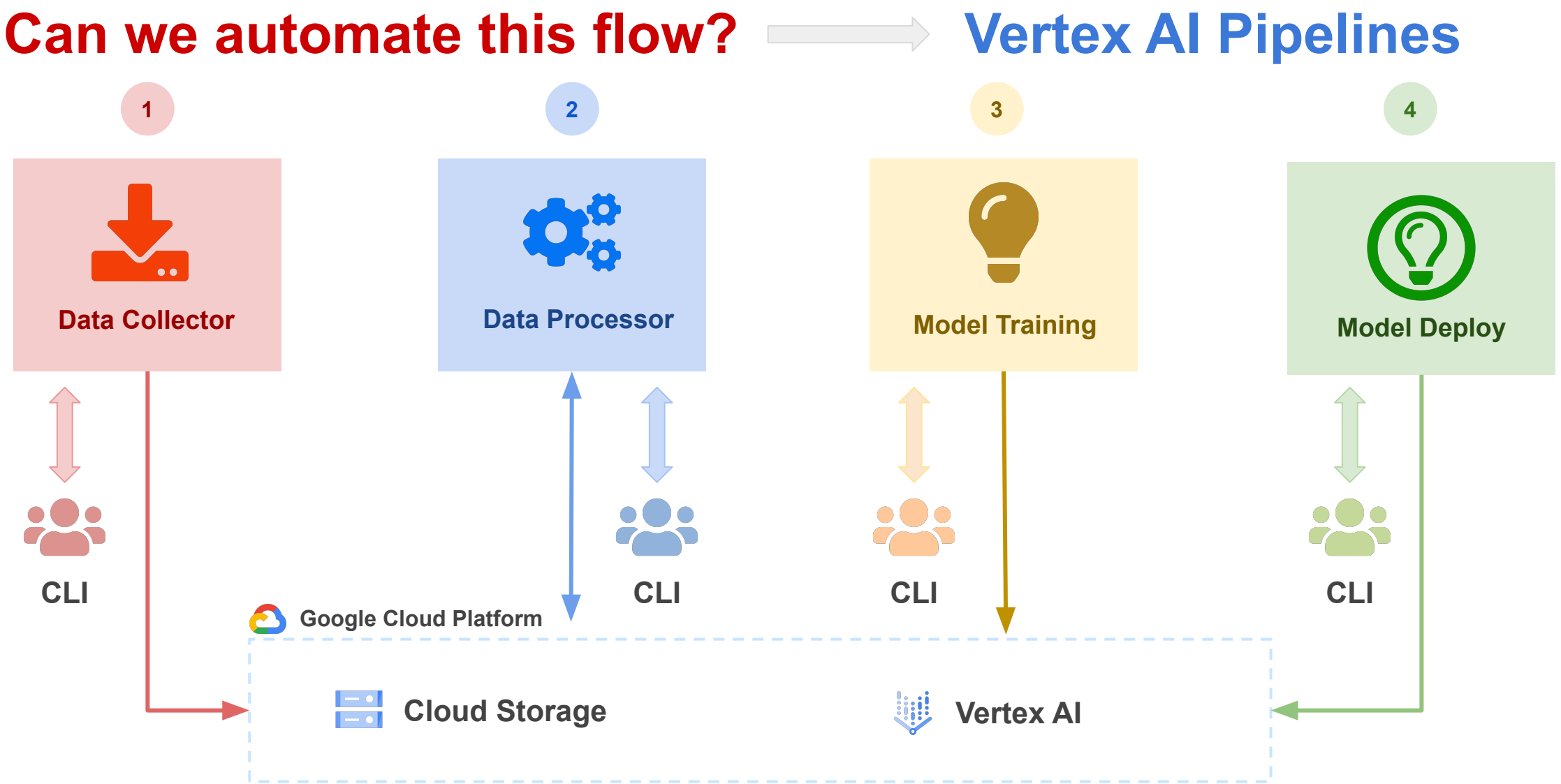
From Deployment to Workflow Automation

- So far, we've deployed a single model — manually, step by step.
- In production, we need **automation** — a repeatable, traceable sequence that handles data → training → deployment automatically.
- That's where **Vertex AI Pipelines** comes in:
  - It's Google's managed way to orchestrate ML workflows.
  - Built on **Kubeflow**, but fully integrated into GCP.

ML Workflow: Cheese App



ML Workflow: Cheese App



What is ML Workflow Management?

- Coordinates and automates **end-to-end ML steps** — from data collection to deployment.
- Defines **how tasks depend on each other** and ensures they run in the correct order.
- Enables **reproducible, repeatable experiments** — the same code can re-run with new data.
- Handles **scheduling, monitoring, and resource allocation** automatically.

Outline

1. Recap
2. Serverless: Model Deployment
3. ML Workflow Management
   - **Vertex AI Pipelines**

What is Vertex AI Pipelines?

- **Vertex AI Pipelines** automates and orchestrates the entire ML process — from data prep to training and deployment.

- It lets you define each step in **Python**, and executes them **serverlessly**, **reproducibly**, and **at scale**.

- Powered by **Kubeflow**, but fully managed inside GCP — no Kubernetes setup required.

Kubeflow and the DSL

- **Kubeflow** is an open-source system that helps run ML workflows on **Kubernetes**.
- It defines how steps like data prep, training, and deployment connect and run.
- **Vertex AI Pipelines** uses Kubeflow underneath but takes care of all the setup for you.
- You write your workflow in Python using a **DSL** — a simple way to describe steps and how they link together.

*Think of **Kubeflow** as the machine, and the **DSL** as the recipe that tells it what to do.*

Kubeflow and the DSL

- **Kubeflow** is an open-s workflows on **Kubernetes**.
- It defines how steps like ment connect and run.
- **Vertex AI Pipelines** us s care of all the setup for you.
- You write your workflow ble way to describe steps and how they link

*Think of **Kubeflow** as the r ipe that tells it what to do.*



What is a DSL?

DSL = Domain-Specific Language

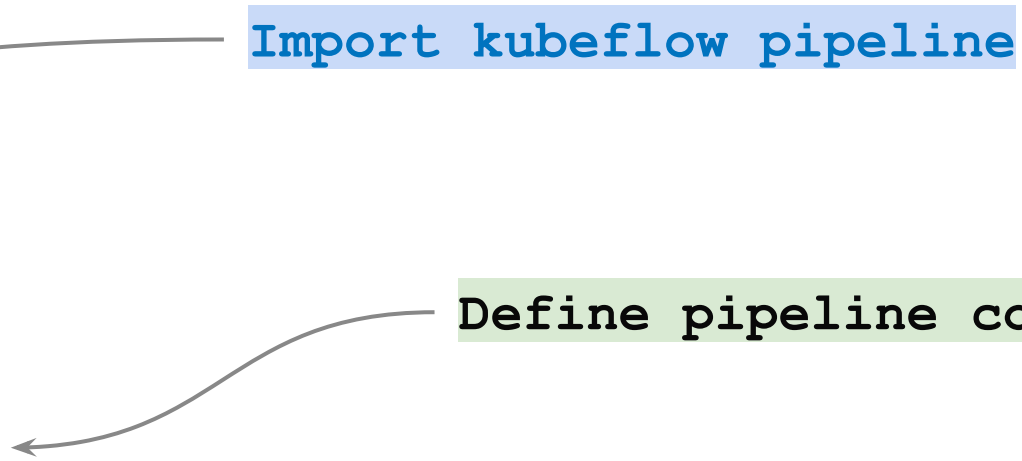
- A **DSL** is a small programming language made for one specific purpose.
- In our case, it's for describing **machine-learning pipelines**.
- We write it in **Python**, using a simple syntax that says what each step does and how they connect.
- The DSL makes complex workflows easy to read, share, and reuse.

Remember:

The DSL is not a new language to learn — it's just a simpler, Python-based way to say, *"Here's how to make the cheese."*

Building Vertex AI Pipelines

```
# Import Kubeflow Pipelines
from kfp import dsl
# Define Components
@dsl.component
def square(x: float) -> float:
    return x**2
@dsl.component
def add(x: float, y: float) -> float:
    return x + y
@dsl.component
def square_root(x: float) -> float:
    return x**0.5
# Define Pipeline
```



Building Vertex AI Pipelines

```
...

# Define Pipeline
@dsl.pipeline
def sample_pipeline(a: float = 3.0, b: float = 4.0) -> float:
    a_sq_task = square(x=a)
    b_sq_task = square(x=b)
    sum_task = add(x=a_sq_task.output, y=b_sq_task.output)
    return square_root(x=sum_task.output).output

# Build yaml file for pipeline
compiler.Compiler().compile(
    sample_pipeline, package_path="sample-pipeline.yaml"
)
```

Define Pipeline, an orchestration of how you want your component tasks to run

Compile pipeline into a yaml file

From DSL to Running the Pipeline

How it works

1. You define your workflow in **Python** using the **KFP DSL** (kfp.dsl).

2. The DSL compiler creates a **YAML file** — this file describes the pipeline and all its steps.

3. You use the **Google Cloud SDK** (google.cloud.aiplatform) to send that YAML to Vertex AI.

4. Vertex AI then runs the full workflow — serverlessly — step by step.

Analogy:

The DSL writes the recipe, the YAML is the printed recipe card, and the SDK is the chef who follows it.

From DSL to Running the Pipeline

How it works

1. You define your workflo (kfp.dsl).
2. The DSL compiler creat bes the pipeline and all its steps.
3. You use the **Google Cl** m) to send that YAML to Vertex AI.
4. Vertex AI then runs the tep by step.

Analogy:

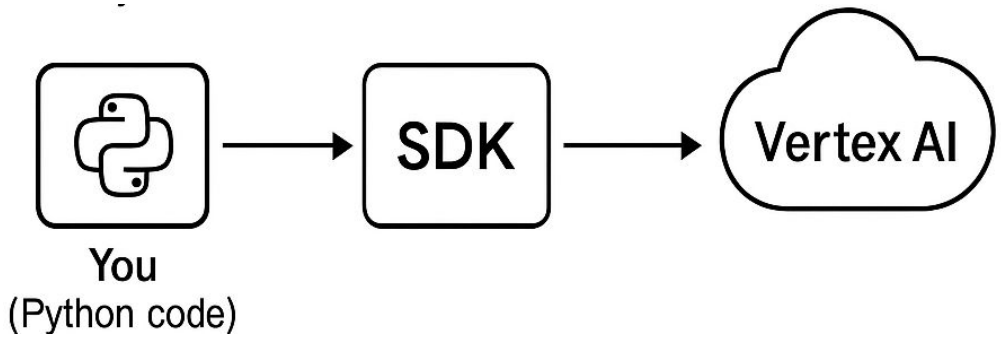
The DSL writes the recipe, ard, and the SDK is the chef who follows it.



What is SDK



- **SDK** stands for *Software Development Kit.*
- It's a **set of tools and libraries** that let you control a platform from your own code.
- The **Google Cloud AI Platform SDK** lets you:
  - Create and manage datasets, models, and pipelines
  - Submit jobs and run workflows on Vertex AI
  - Automate what you would normally do in the web console
- In short: **the SDK is your remote control for Vertex AI.**



Running Vertex AI Pipelines

```
# Initialize GCP
from import google.cloud.aiplatform import aip


# Create a Pipeline Job in Vertex AI
job = aip.PipelineJob(
    display_name=DISPLAY_NAME,
    template_path="sample-pipeline1.yaml",
    pipeline_root=PIPELINE_ROOT,
    enable_caching=False,
)

# Run the Pipeline Job
job.run()
```

Import Google Cloud SDK

Create a pipeline job

Run pipeline job in Vertex AI

Building Vertex AI Pipelines

Steps to build pipelines for your custom containers

1. **Make your containers callable**
   - Each step (data prep, train, deploy) runs inside a container.
   - The container must accept inputs and produce outputs.

2. **Build & push images**
   - Push your container images to **Artifact Registry** or **Container Registry** so Vertex AI can access them.

3. **Define workflow structure**
   In **kfp.dsl**, you describe how these steps connect — that structure is called a **DAG (Directed Acyclic Graph)**.
   - The DAG defines the order: *what runs first, what depends on what.*

Making Container Callable

Dockerfile

```
# Use the official Debian-hosted ...
FROM python:3.9-slim-buster
.
.
.
# Add the rest of the source code.
RUN --chown=app:app . /app
# Entry point
ENTRYPOINT ["uv", "shell"]
```

Dockerfile

```
# Use the official Debian-hosted ...
FROM python:3.9-slim-buster
.
.
.
# Add the rest of the source code.
RUN --chown=app:app . /app
# Entry point
ENTRYPOINT ["/bin/bash","./docker-entrypoint.sh"]
```



Making Container Callable

Development mode:
Authenticated to GCP
run a shell to test cli inside container

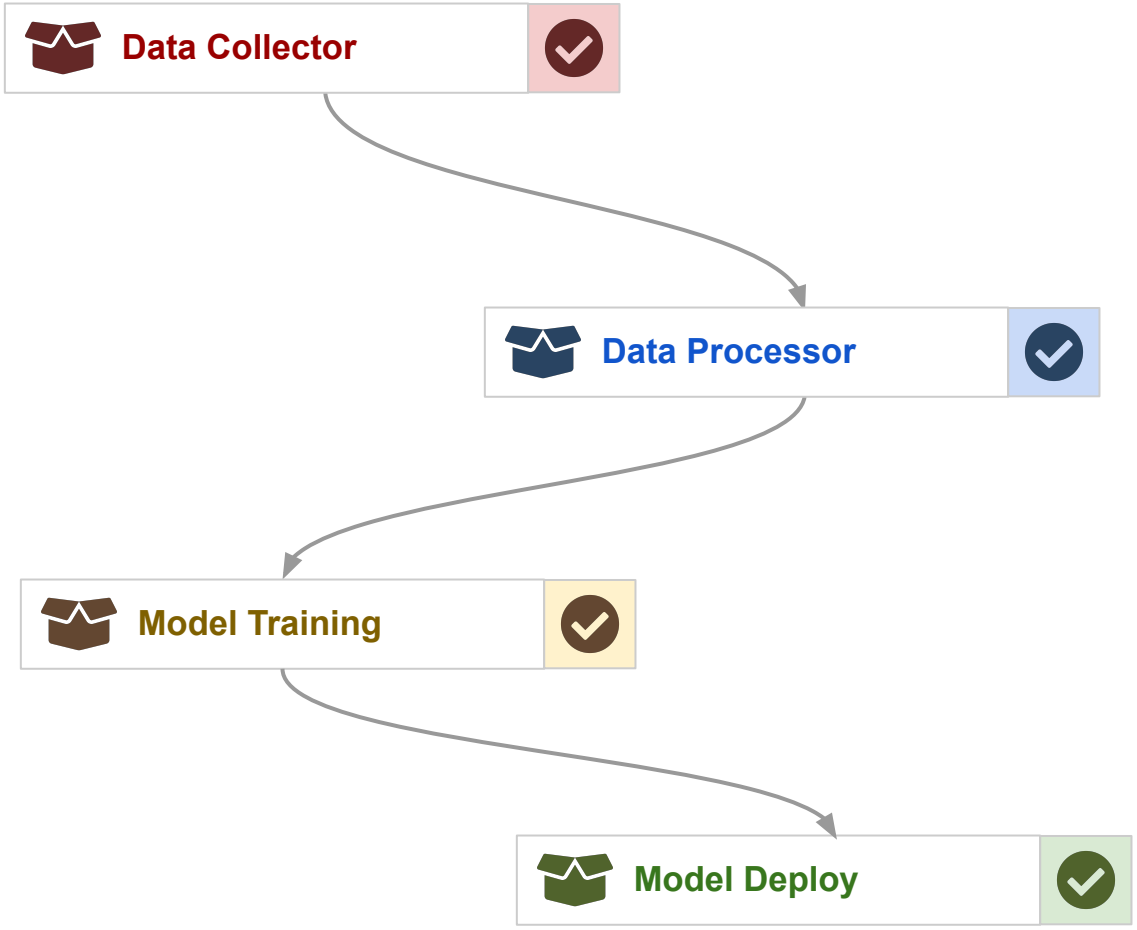
docker-entrypoint.sh

```
#!/bin/bash
args = "$@"

if [[ -z ${args} ]];
then
    # Authenticate gcloud using service account
    gcloud auth activate-service-account --key-file $GOOGLE_APPLICATION_CREDENTIALS
    # Set GCP Project Details
    gcloud config set project $GCP_PROJECT
    uv shell
else
  uv run python $args
fi
```

Production mode:
Run container using "docker run ... cli.py –search"

Building Vertex AI Pipelines



1. Download images
2. Uploads to GCP

1. Verify images
2. Check for duplicates
3. Convert to TF Records

1. Train model
2. Save model

1. Upload to Registry
2. Deploy model Endpoint

Building Vertex AI Pipelines



1. Download images
2. Uploads to GCP

1. Verify images
2. Check for duplicates
3. Convert to TF Records

1. Train model
2. Save model

1. Upload to Registry
2. Deploy model Endpoint

Tutorial: Vertex AI Pipelines

Steps to build **Vertex AI Pipelines** on the cheese app ML workflow components:



- Make Containers Callable.
- Build & Push Image.
- Build ML Pipeline.
- Run Pipeline in Vertex AI
- For detailed instructions, please refer to the following link
  - Cheese App Workflows. (https://github.com/dlops-io/ml-workflow#cheese-app-ml-workflow-management)
  - View Vertex AI Pipelines. (https://console.cloud.google.com/vertex-ai/pipelines/runs)

Putting It All Together — From Recipe to Factory

How everything connects

1. **DSL** → you *define the workflow* in Python (your **recipe**)
2. **Compiler** → turns it into a **YAML file** (the **printed instructions**)
3. **SDK** → sends that YAML to **Vertex AI** (the **factory operator**)
4. **Kubeflow** → the **engine** that actually runs the workflow
5. **Vertex AI Pipelines** → manages the process *serverlessly*, step-by-step

💡 *Each container is a worker in the factory — the DAG tells them what to do and in what order.*


DELETE!!


#FALLONTONIGHT
Students
Prof. Pavlos